



AI-Powered Debt Relief & Financial Recovery Platform

Phase 2 — Requirement Analysis

Technology stack, customer journey, and data flow architecture
for the GenAI-driven settlement negotiation platform

Prepared as part of the project documentation set
Document type: Technical Requirements & Architecture Overview

Phase 2: Requirement Analysis

This document records the requirements, technical choices, customer journey, and data flow architecture for the BlogForge platform — an AI-powered tool that helps borrowers assess delinquent debt and generate lender-ready hardship settlement proposals.

1. Technology Stack Selection

The platform is built on a modern, scalable, and responsive architecture, spanning frontend, backend, data, and generative AI layers.



Figure 1. Layered technology stack, from UI to the Gemini AI layer.

Layer	Technology	Purpose
Frontend	React.js (Vite)	Quick build tool and robust Single Page Application (SPA) structure.
	Vanilla CSS	Premium dark-mode token system using HSL configuration, custom animation, and glassmorphic card patterns.
	Lucide-React	Premium, unified vector iconography.
Backend	FastAPI (Python)	High-performance, async-enabled REST API with auto-generated Swagger documentation.
	Uvicorn	Lightweight ASGI web server.

Layer	Technology	Purpose
Database & ORM	SQLite	Lightweight, serverless relational database for local development and testing.
	SQLAlchemy ORM	Handles SQL queries via Python models; maps easily to cloud Postgres (Neon / Supabase) in production.
Generative AI Layer	Google Gemini API (gemini-1.5-flash)	Generates intelligent, professional, lender-specific hardship negotiation letters.

Why this stack works well for an MVP: Vite + React gives fast iteration for the UI; FastAPI's async support and automatic Swagger docs make backend testing and handoff easier; SQLite keeps local development frictionless while SQLAlchemy's ORM abstraction means the same code can point at a managed Postgres instance (e.g. Neon or Supabase) with minimal changes when the platform moves to production.

2. Customer Journey Map

This journey traces how a borrower, John Doe, uses the platform to resolve a credit card delinquency, from first discovering the tool through to mailing a negotiated settlement offer.

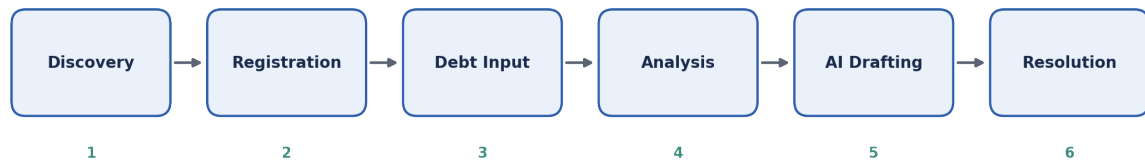


Figure 2. End-to-end journey from discovery to resolution.

1**Discovery**

John is stressed by collection letters and searches for an automated debt settlement tool. He finds BlogForge.

2**Registration**

John signs up, entering his monthly gross income and basic expenses.

3**Debt Input**

He enters his delinquent Chase Credit Card details: balance (\$10,000), monthly EMI (\$350), and 6 overdue months.

4**Analysis (Settlement Predictor)**

John views his dashboard. A visual meter shows a "Severe" stress rating due to a high DTI. The predictor suggests a 35% settlement (\$3,500 payoff) based on the 6-month delinquency.

5**AI Drafting**

John selects the Chase Card, chooses "Job Loss" as his hardship, adds salary details, and clicks "Generate." Gemini AI drafts a formal proposal.

6**Resolution**

John copies the letter, mails it to Chase's settlement department, and begins paying off the settled amount in structured installments.

3. Data Flow Diagram (DFD)

3.1 Level 0 DFD — Context Diagram

The context diagram shows the platform as a single process exchanging data with the borrower, the Gemini AI API, and the persistence layer.

Level 0 DFD — Context Diagram

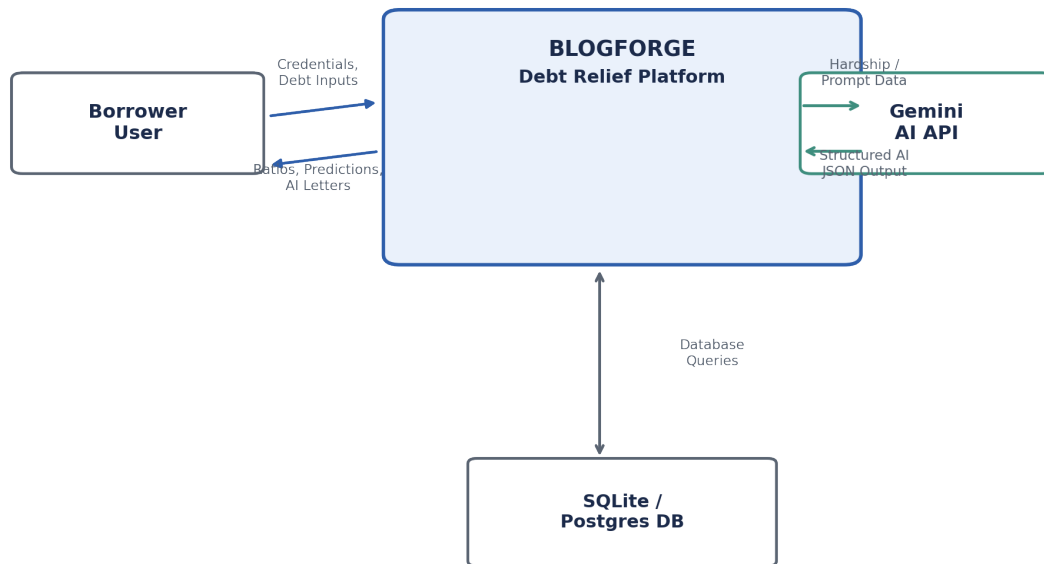
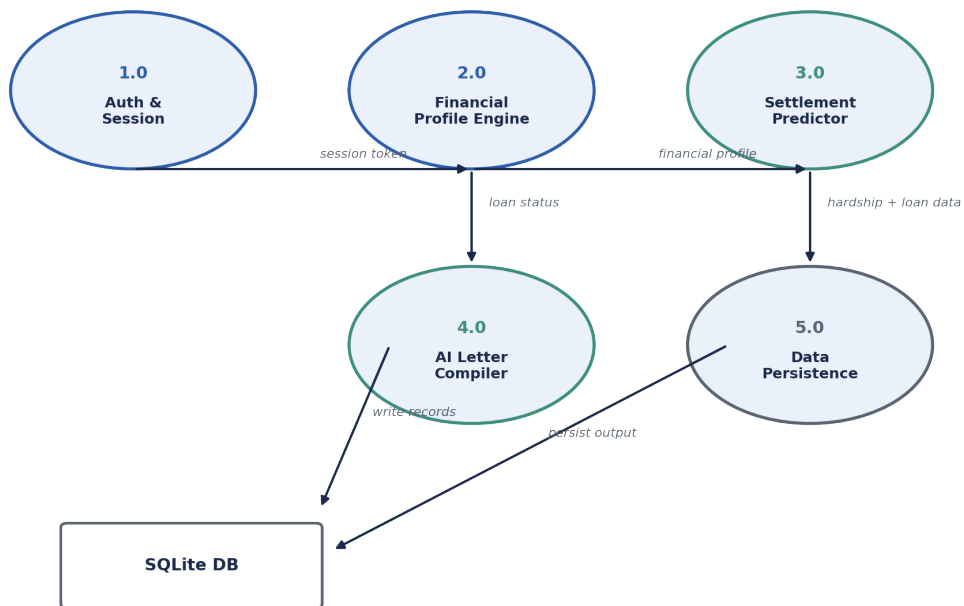


Figure 3. Level 0 context diagram of the BlogForge platform.

3.2 Level 1 DFD — Functional Decomposition

The platform's core logic decomposes into five processes, from authentication through to persisting negotiation history.

Level 1 DFD — Functional Decomposition*Figure 4. Level 1 functional decomposition of platform processes.*

Process	Description
1.0 — Auth & Session	Verifies user registration and login credentials, encoding active session state inside JWT security tokens.
2.0 — Financial Profile Engine	Takes user income, expenses, and total loan EMIs to calculate surplus and update DTI and EMI ratios.
3.0 — Settlement Predictor	Evaluates loan type and overdue months against user stress levels to output a suggested settlement percentage.
4.0 — AI Letter Compiler	Sends the user profile, loan status, and hardship details to the Gemini API, returning a strategy and drafted letter.
5.0 — Data Persistence	Writes user inputs, predictions, and negotiation history to the SQLite database.

4. Additional Notes & Recommendations

The following points are not part of the original requirement text, but are common considerations worth tracking for a platform of this kind as it moves past MVP stage.

4.1 Security & Compliance Considerations

- Encrypt sensitive financial data (income, EMI, balances) at rest, not just in transit.
- Rotate and scope JWT secrets separately per environment; keep token lifetimes short with refresh support.
- Apply rate limiting on the AI Letter Compiler endpoint to control Gemini API cost and abuse risk.
- Log negotiation history for audit purposes, but mask or redact PII in application-level logs.
- Review regional consumer-debt and financial-advice regulations before offering settlement percentages as guidance.

4.2 Scaling from SQLite to Production

- SQLAlchemy's ORM layer means switching the connection string to Postgres (Neon/Supabase) requires minimal code change.
- Add a migration tool (Alembic) early, even during SQLite development, to avoid a painful schema-migration rewrite later.
- Introduce connection pooling once concurrent users grow beyond what SQLite's single-writer model handles well.

4.3 Suggested Next Steps

- Phase 3: Detailed UI/UX wireframes for the dashboard, stress-rating meter, and letter-generation flow.
- Phase 4: API contract definition (OpenAPI schema) between the FastAPI backend and the React frontend.
- Phase 5: Prompt-engineering pass on the Gemini letter-generation prompt, with test cases per hardship type (job loss, medical, divorce, etc.).
- Ongoing: Track DTI/EMI calculation accuracy against real-world settlement outcomes to refine the Settlement Predictor.